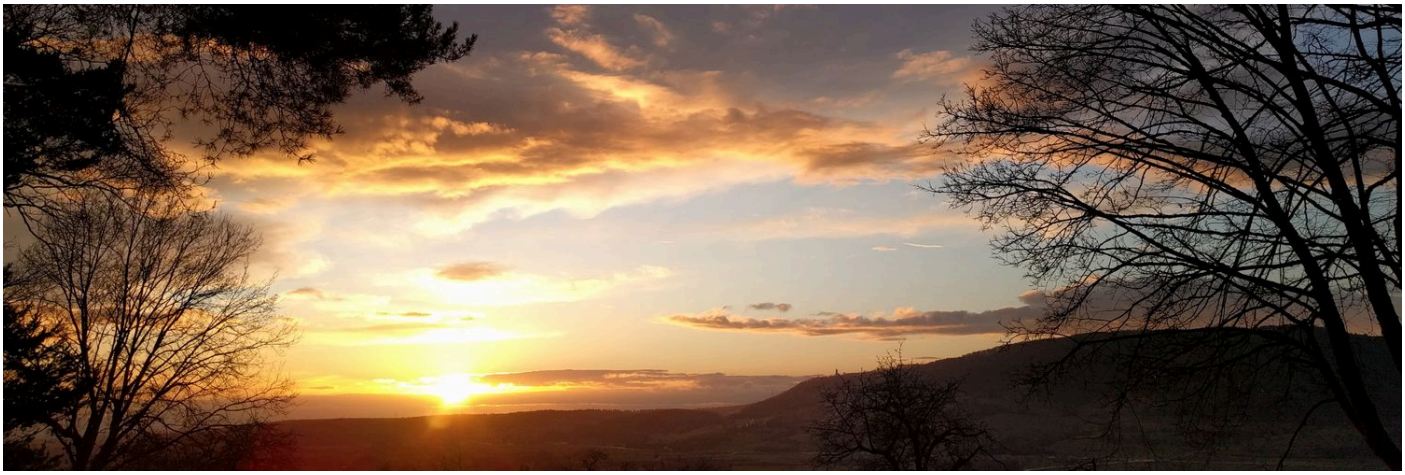
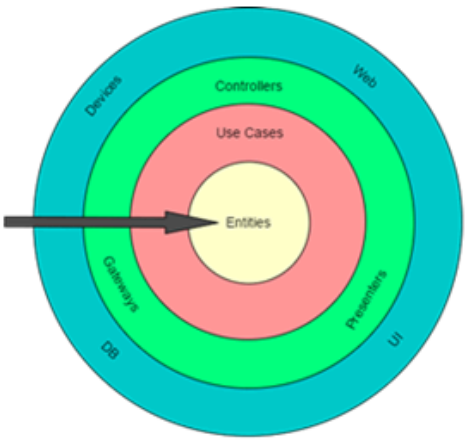
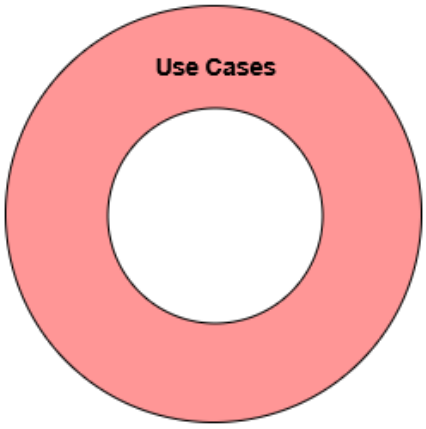


Plainionist

Implementing Clean Architecture - What is a use case?

11 Feb 2018







Now that my architecture is screaming the business capabilities of my system let's look at those with more detail.

In the Clean Architecture all the application specific business rules go into the use cases circle.

But what is a use case? How big should it be? How does it interact with its environment?

Read on!

Definitions

As a starting point for answering these questions I like to fish for some definitions ...

Wikipedia:

> In software and systems engineering, a use case is a list of actions or event steps typically defining the interactions between a role (known in the Unified Modeling Language as an actor) and a system to achieve a goal.

Clean Architecture book:

> These use cases orchestrate the flow of data to and from the entities, and direct those entities to use their Critical Business Rules to achieve the goals of the use case.

OK, these definitions are rather high-level and nothing concrete. I kinda expected that ;-)

Enough of theory - let's look at something more practical ...

The example

To recap, *Athena* is an "agile backlog visualizer" implemented in F# using Asp.Net MVC. It provides various reports to improve the overall transparency in a software project. One major report is showing the ranked backlog with cut-lines:

Clinical/Define All **Architect** All

Id	All	All	Rank	Estimation	Area Path	All	Title
2316472	APL	MMP	20	0	Path\ALG	Nick	provide improved landmark algorithm
2622469	APL	MMP	20	0	Path\ALG	Nick	extend new table removal
2316865	APL	MMP	22	0	Path\ALG	Nick	establish intelligent radial range
2316896	APL	MMP	23	0	Path\ALG	Nick	build extended rib unfolding
2318871	APL	MMP	24	0	Path\ALG	Ryan	construct new body registration
2002042	APL	MMP	50	185	Path\IBR	Nick	build automatic table removal
2289256	APL	MMP	53	80	Path\ABD	Matthew	improve automatic table removal
2291269	CPL	MMP	54	170	Path\IBR	Nick	create advanced table removal
2288375	APL	MMP	62	53	Path\ABD	David	generate intelligent tumor detection
2288820	CPL	MMP	164	34	Path\YIK	David	make new body registration
2289254	APL	MMP	254	0	Path\ABD	Matthew	make automatic table removal
2289257	APL	MMP	258	88 ???	Path\ABD	Matthew	construct revised radial range
2289258	APL	MMP	260	0	Path\ABD	Matthew	enhance improved distance measurement
2289259	APL	MMP	262	25	Path\ABD	Matthew	extend automatic table removal
2313175	APL	MMP	360	50	Path\SLS	Nick	construct improved table removal
2149234	CPL	MMP	370	175	Path\SLS	Nick	create intelligent landmark algorithm
2265280	CPL	MMP	390	20	Path\SLS	Nick	generate intelligent vessel segmentation
1629855	CPL	BestEffort	1050	80	Path\IBR	Nick	make revised landmark algorithm
2331194	CPL	BestEffort	1051	!!	Path\IBR	Nick	provide intelligent landmark algorithm
1649156	PPL	BestEffort	1100	60	Path\SLS	Nick	generate new table removal
2288386	APL	BestEffort	1174	???	Path\YIK	David	extend advanced table removal
2288392	APL	BestEffort	1180	???	Path\YIK	David	prepare extended distance measurement
2574658	CPL	BestEffort	1187	???	Path\YIK	David	generate upgraded vessel segmentation
2289262	APL	BestEffort	1254	???	Path\ABD	Matthew	make revised rib unfolding
2289263	APL	BestEffort	1256	???	Path\ABD	Matthew	make extended tumor detection
2289264	APL	BestEffort	1258	???	Path\ABD	Matthew	provide advanced body registration
2289266	APL	BestEffort	1262	???	Path\ABD	Matthew	create intelligent vessel segmentation
2289267	APL	BestEffort	1264	???	Path\ABD	Matthew	establish extended rib unfolding
1659622	PPL	BestEffort	1266	148 ???	Path\ABD	Matthew	enhance advanced rib unfolding
2316501	APL	BestEffort	1300	0	Path\ALG	Nick	make automatic vessel segmentation
1962809	PPL	BestEffort	1370	124	Path\JSS	Nick	make advanced tumor detection
2265313	APL	BestEffort	1400	???	Path\SLS	Nick	enhance intelligent body registration

Is providing this report a use case? On the most highest and abstract level: yes. The simplified “list of actions” is:

1. Get all work items from TFS (Team Foundation Server)
2. Rank all work items
3. Get the team capacity
4. Determine the cut-lines

5. Generate the report (including all further details like TFS links and hints for missing information)

This would probably be the level on which I would specify an expected system behavior.

But should I implement this use case in a single use case interactor?

How big should a use case be?

There is no ideal size of a use case. Use cases can be long or short, high-level or more concrete. It all depends on the level of abstraction we want to consider.

Let me rephrase the question: How big should a use case INTERACTOR be?

So what would be a driving principle for deciding about “size”? Correct: Single Responsibility Pattern (SRP).

Let’s look closer at the list of actions from above …

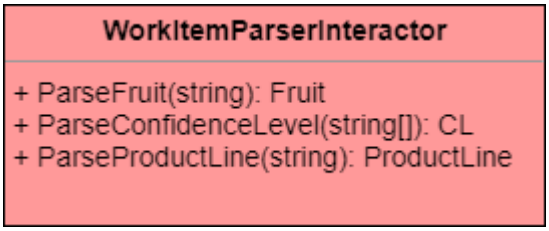
Action: Get all work items from TFS

This is clearly data access, isn’t it?

Yes and no. Obviously fetching the (raw) data from TFS is data access. But there are also business rules. The TFS itself is rather generic. We have put quite some custom conventions on top of it like: fruits, confidence levels, product lines and many more.

Interpreting plain text fields and generic tags and giving these information business semantics is about business rules. So I want to have this logic in the use cases circle. Guided by the SRP I would say

⇨ First use case interactor found

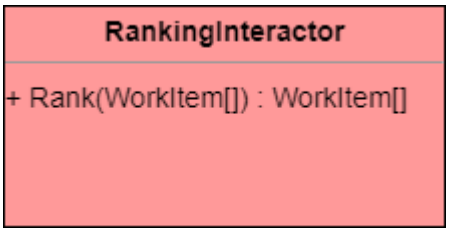


Action: Rank all work items

Ranking the backlog is definitively about business rules as those define how the ranking should happen. Ideally this is a very simple task which just sorts work items by stack rank. But reality is rarely ideal so I could imagine additional rules, e.g.:

- Always rank MMP higher than “best effort”
- Rank work items without any given rank lowest
- In case of duplicate ranks, rank one product line over the other product line.

⇨ As this has nothing to do with backlog conventions and parsing I will add a second interactor.



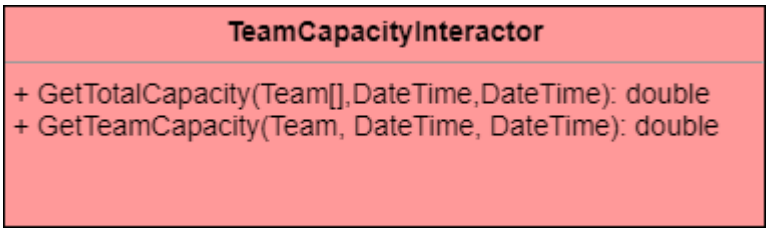
Action: Get the team capacity

This is again mostly about data access. We have a non-TFS data source where all teams are modeled with their ramp up and ramp down in head count for one year. Getting this (raw) data is pure data access.

In a second step I need to calculate the team capacity from the given data. This step is again about business rules:

- Subtract corridors like hot fixes for the installed base from the modeled head count
- Convert head count into person days by applying an “availability factor” (e.g. remove holidays)
- Apply correct time frame (e.g. duration of an iteration or software version)

⇨ As this involves quite different entities than the previous two interactors I will create a third one.



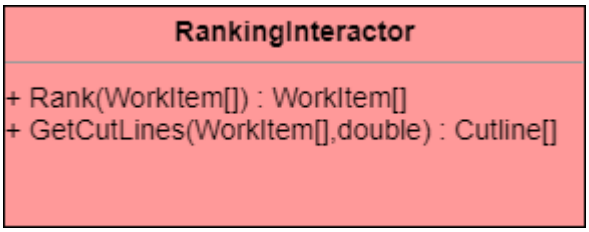
Action: Determine the cut-lines

Now that I have a ranked backlog and the capacity I can determine the cut-lines.

> *Note:* You may wonder why I talk about multiple cut-lines. This comes from the confidence levels. Each confidence level defines a certain percentage by which the actual estimation is correct, e.g. confidence level "low" could be defined as "+/- 50%" which means that the actual estimation could be "50% too expensive" or "50% too cheap". This approach basically gives three numbers for each estimation: an optimistic, a pessimistic and a realistic one. Hence, we have three cut-lines.

Calculating the cut-lines is a simple algorithm which walks the backlog from top to bottom, sums up estimations and matches these against the capacity. It is clearly about business rules so I will add it to a use case interactor.

⇨ Considering the existing interactors and my preference of pragmatic decisions I will put this logic into the `RankingInteractor`.



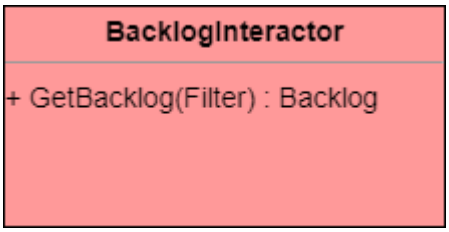
Action: Generate the report

At first glance this may look like "UI work" but that's not all. Here are further business rules to consider:

- Validate missing information (missing confidence level, missing estimation)
- Provide total capacity for reference
- Provide total effort for reference
- Compute list of involved team members (for the filters)

I want to have these business rules realized in an interactor as well.

⇨ Even with my preference for pragmatic decisions this logic doesn't fit nicely into any existing interactor without violating SRP. So let me create a new one.



How do I combine use case interactors?

We have created four interactors so far. But how do we "combine" them to realize the high-level use case?

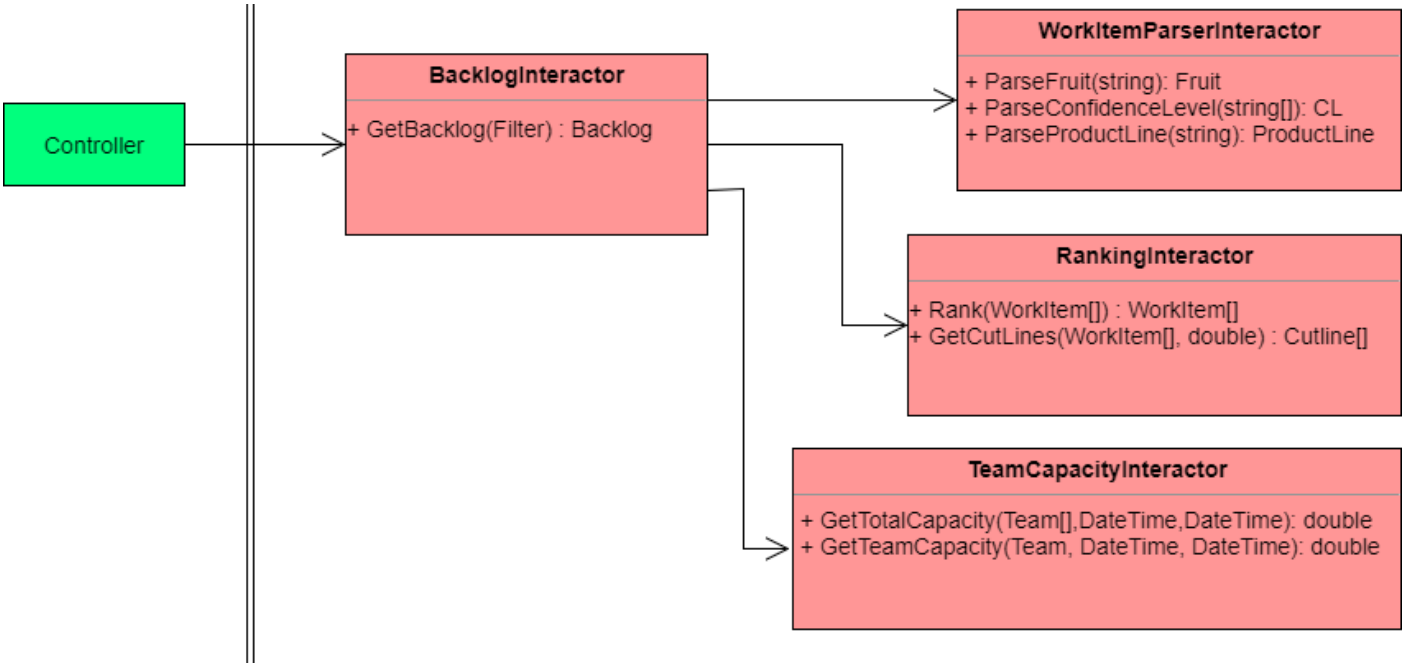
Basically there are two possibilities:

1. Combine the interactors in the controller, so in the interface adapters circle
2. Combine the interactors in another interactor, so in the use cases circle

I could easily find pros and cons for each alternative. The driving question for me would be, how much (business) logic is involved in combining the other interactors?

I want the interface adapters (controllers, presenters) rather dumb. So as soon as some logic is involved in combining interactors I prefer having another interactor realizing the combination.

For the use case discussed here I tend to make again a pragmatic decision: I will make the `BacklogInteractor` the "combining interactor" which calls the other interactors.

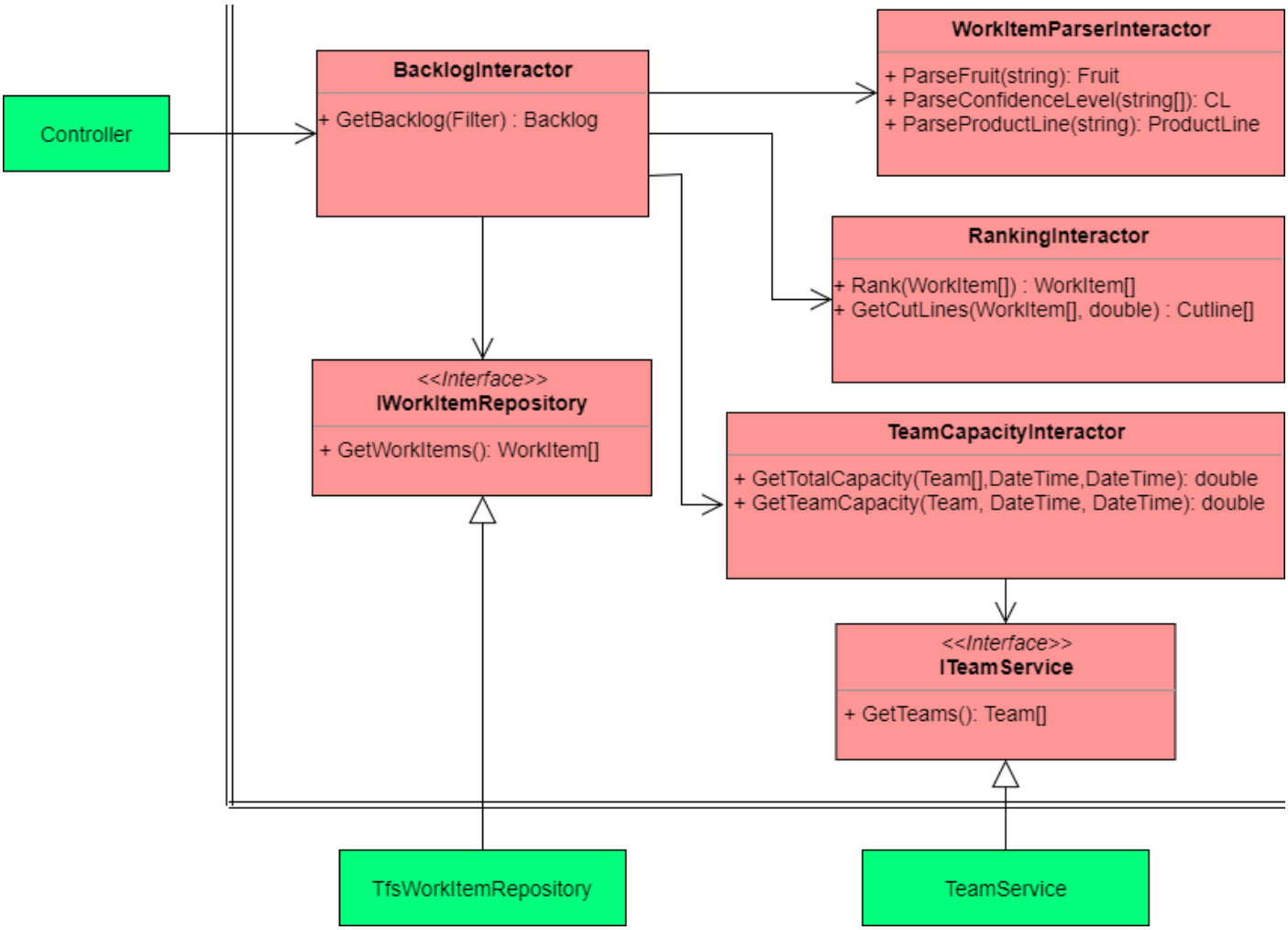


How do I access the database?

In his book Uncle Bob writes about database access:

> Between the use case interactors and the database are the database gateways. These gateways are polymorphic interfaces that contain methods for every create, read, update, or delete operation that can be performed by the application on the database. For example, if the application needs to know the last names of all the users who logged in yesterday, then the UserGateway interface will have a method named getLastNamesOfUsersWhoLoggedInAfter that takes a Date as its argument and returns a list of last names.

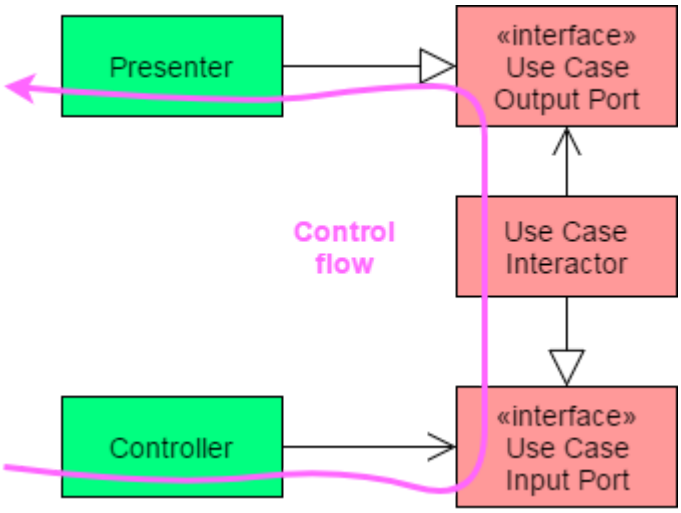
For our use case I will define two interfaces. One to get the work items from TFS and one to get the team capacity information from the external service.



Details about accessing “IO devices” and external systems I will discuss in one of my next posts.

How do I interact with controller and presenter?

According to the Dependency Rule a use case interactor must not depend on a controller or presenter. Instead, the use case interactor defines “input ports” and “output ports” to invert the dependencies.



In the use case I have described here the setup is simpler.

- As I use Asp.Net MVC, the controller and the presenter are the same class: the Asp.Net MVC controller.
- All methods I have defined on the interactors so far are simple and pure functions which get parameters and return results. So I don't need to define any "ports". I will simply pass DTOs (data transfer objects) around.

So I will just let the controller call `BacklogInteractor.GetBacklog` passing some filter DTO which will return some report DTO as result.

Which role is than actually left to the controller and the presenter? The answer to this question I will leave to another post.

How do others think about use cases and interactors?

During research for this post I found many discussions about the "right cut" of use cases. Here is a list of some well crafted thoughts:

- How big or small should a Use Case Interactor be in Clean Architecture?
- Do Interactors in clean architecture violate the Single Responsibility Principle?
- Who should handle combining interactors?

The "Implementing Clean Architecture" series

- How to implement the Clean Architecture?
- Implementing Clean Architecture - An Overview
- Implementing Clean Architecture - Make it scream
- Implementing Clean Architecture - What is a use case?
- Implementing Clean Architecture - Of controllers and presenters
- Implementing Clean Architecture - Are Asp.Net controllers "Clean"?
- Implementing Clean Architecture - Frameworks vs. Libraries
- Implementing Clean Architecture - Case Study: Sending e-mails
- Implementing Clean Architecture - To use or not to use MediatR?

Tags: `clean-architecture`

Related Posts

Implementing Clean Architecture - To use or not to use MediatR? 20 Jan 2023
Implementing Clean Architecture - Case Study: Sending e-mails 02 Feb 2022
Implementing Clean Architecture - Frameworks vs. Libraries 01 May 2019
Implementing Clean Architecture - Are Asp.Net controllers "Clean"? 18 Mar 2018

ALSO ON **BLOG**

Implementing Clean Architecture - ...
8 years ago • 4 comments
Clean Architecture is screaming by focusing on its core business purpose ...

Implementing Clean Architecture - Are ...
8 years ago • 4 comments
In the Clean Architecture controllers belong to the “interface adapters” and ...

Book Review: Atomic Habits
4 years ago • 2 comments
We all want to get rid of some bad habits and we all want to get better at ...

Imple Archit
7 years a
In Clear Depend usage of

14 Comments Login ▾

G Join the discussion...

LOG IN WITH OR SIGN UP WITH DISQUS ?

Name

♡ 2 Share Best Newest Oldest



Admirador de Damas
4 months ago

Clean Architecture has a single rule: dependencies must always point toward the domain.

Many NestJS projects violate this rule, even those that claim to use Clean Architecture.

NestJS has a key feature that is often misused and ends up breaking the dependency rule.

That same feature, when used correctly (as shown in the documentation), allows the rule to be respected without hacks.

The key is to properly understand the rule and know what to do and what not to do. The documentation explains how to apply the correct option.

There are four strategies to implement transaction management in a NestJS project that uses Clean Architecture.

One of these strategies relies on a feature that has existed since Node.js 12.

Each strategy can be analyzed in terms of:

* Whether it violates Clean Architecture
* Whether it is coupled to the framework or a specific library
* The complexity of the required changes
* Whether it facilitates testing or not
* Whether it is thread-safe

0 0 Reply

Zeki Gulser
7 years ago

That's a good post. I've one addition though. Maybe this is my misunderstanding but I think interactors are just layers to provide implementations to the underlying business rules (either over domain objects or use case objects). They don't really implement the use cases or include gateways. They may provide some coordination code and then they just delegate the calls

usually using command pattern to actually execute the request.

0 0 Reply



plainionist Mod → Zeki Gulser

7 years ago

Thx for sharing your thoughts. As to my understanding of Uncle Bobs Clean Architecture "interactor" is the name he has chosen for these classes which implement the use cases of the application. Use cases control the "dance of the Entities" which are representing the core business rules of the application or enterprise. For that purpose use cases use "gateways" (repositories) behind interfaces defined on the use case layer to access the data needed.

1 0 Reply

Z

Zeki Gulser → plainionist

7 years ago

That part I get but then it confuses me at the point where interactors should contain only application specific rules. If we include gateways into it, then I think we would also tie the business logic into it which we should not.

I guess the the name also implies that. It s an interactor for the app layer and it delegates requests to implementors in the domain layer through polymorphically deployed boundary classes.

0 0 Reply



plainionist Mod → Zeki Gulser

7 years ago

I guess the confusion comes from the fact that in software (architecture) many terms are (re)used in different contexts with slightly different meaning. In Clean Architecture Uncle Bob clearly defines "use case interactors" as the place where "application specific business rules" located (https://blog.cleancoder.com.... Gateways (e.g. Repositories) are located outside the use case layer but implement interfaces defined by the interactors and located in the use case layer. With this approach business logic (in interactors) remain independent from "technical details".

1 0 Reply

Z

Zeki Gulser → plainionist

7 years ago

This is exactly what I'm saying but only by 'repository' I meant the interactor :) (maybe because I'm an android developer) and that's what confused me apparently.

0 0 Reply



plainionist Mod → Zeki Gulser

7 years ago

Great that we could sort that out. thx for the discussion :)

0 0 Reply



Jihwan Kim

8 years ago

Interactor calling other interactors? this is simply WRONG.

0 0 Reply

B **Beb** Jihwan Kim

7 years ago

Could you at least explain why instead saying this is wrong.
It won't help..

2 0 Reply



Jihwan Kim Beb

7 years ago

Because you just broke the dependency graph which should b flattened not deepened. Now u just made interactor A ccontaining/calling another/other interactors B,C,etc., dependency must b maintained that way. And you better remember the rule and the tangled relationship or u will trap urself in Circular dependency problems. For simple solution, do not interactors depends on each other. Take the duplicated logic in sperate class then manipulate the class from many interactors. Same rule goes for other classes such as Repository.

1 0 Reply



Matthew Jacobi Jihwan Kim

7 years ago

All it is is a facade to simplify the other interactors. It's not necessarily wrong since the facade depends on the interactors it wraps it hasn't broken the graph.

1 0 Reply



plainionist Mod Jihwan Kim

7 years ago

thx for sharing ur thoughts!
I agree that coupling use cases also adds complexity - as every kind of reuse does. Extracting shared logic keeps use cases independent - but only on a "syntactical" level. On semantical level use cases using same shared code also have some kind of dependency: if the shared code is changed for the sake of one use case there is a potential impact on the other use cases. So in any case reuse has to be designed with care.
Finally I think "it depends" as usual when it comes to software design: If I have a small system with about 10 use cases I think reusing use cases directly could be a pragmatic solution with small

risk. On the other hand, in a big system with e.g. 50 use cases I would probably favor ur proposal and extract shared logic.

o o Reply